Path-of-Thought: A Neuron-Chain Knowledge Representation System with Parallel Wavefront Recognition

Jennifer Pearl Volunteer Scientist, University of Houston Center for Nuclear Receptors and Cell Signaling ORCID: 0009-0006-6083-384X jenpearl5@gmail.com

Date: April 6, 2026 (revised) **Original draft:** March 24, 2026

Keywords: cognitive architecture, knowledge representation, path-of-thought, parallel wavefront recognition, neuron-chain graph, LLM steering, persistent memory, cortex-cerebellum architecture, resource allocation, AI training efficiency

License: CC BY-NC-ND 4.0

Source code: https://github.com/LunarFawn/SaraBrain

Note on Method: The author has dyslexia and high-functioning autism — language disabilities affecting written expression. The technical thinking, research, architecture, and all intellectual content are entirely the author's. Claude (an LLM, Anthropic) was used as assistive technology to translate technical reasoning into structured prose. This is a disability accommodation protected under the Americans with Disabilities Act (Titles II and III), Section 504 of the Rehabilitation Act, and the Department of Justice's 2024 guidance affirming AI tools used as assistive technology fall under ADA protections. Use of an LLM as a writing accommodation is equivalent to use of text-to-speech software by a blind researcher — it compensates for a disability, it does not replace the thinking.

Abstract

We present Sara Brain, a cognitive architecture for artificial intelligence based on the **path-of-thought** model: the thesis that a thought is a path through recorded knowledge, and that recognition is the convergence of independent paths from simultaneous observations. Knowledge is stored as directed neuron-segment chains in a persistent SQLite database with full source-text provenance. Concept recognition is performed by launching parallel wavefronts — one per input property — and identifying concept neurons where multiple independent wavefronts converge. Cross-concept contamination is prevented structurally through concept-specific relation neurons. Knowledge accumulates monotonically under the formula `strength = 1 + ln(1 + traversals)`, modeling biological long-term potentiation without decay. A hardwired

innate layer (SENSORY, STRUCTURAL, RELATIONAL, ETHICAL primitives) provides behavioral constraints enforced at the API level and surviving database reset. We present a novel two-layer cognitive architecture in which a large language model (LLM) functions as stateless sensory cortex and the path-graph store functions as persistent hippocampus and long-term memory. In a first experiment, a 94KB path-graph database containing 77 neurons reliably steered the output of a billion-parameter LLM toward principled, testable, parameterized code — where the same model without the path graph produced hardcoded, untestable, monolithic output for the identical task. In a second experiment, a 500KB path-graph database with 793 neurons transformed a 3-billion-parameter model — the smallest viable coding model — into a system producing domain-expert-level output on planetary physics, a domain outside the model's training specialization. We argue that the AI industry is over-investing in cortex capacity (model size, training data volume) and under-investing in memory architecture, and that LLMs should be trained for language competence rather than factual memorization — facts belong in the cerebellum, not compressed into weights. The architecture has been further applied in a professional computational biology context to steer LLM-generated scientific code. The entire system runs on Python 3.11+ with no dependencies beyond the standard library.

---

1. Introduction

The field of artificial intelligence has spent three decades converging on a single architectural bet: that intelligence emerges from the statistical regularities in large training corpora, encoded in billions of floating-point weight parameters and retrieved through matrix multiplication and attention mechanisms. This bet has produced systems of remarkable practical capability. It has not produced systems that can explain themselves.

The question this work addresses is different. Not "how do we get a machine to produce correct outputs?" but "how does thinking work?" The distinction matters: a system designed to produce correct outputs can produce incorrect ones through mechanisms that are invisible and untraceable. A system designed to model correct thinking produces incorrect outputs through mechanisms that are visible, diagnosable, and fixable.

This idea originates in the 1990s, predating transformers, deep learning, and modern LLMs. The central observation: a thought is a path. You start with what you observe. You travel through what you know. Where paths from different observations meet — that intersection is a conclusion. Every step is traceable. Every conclusion is explainable. Nothing is a black box.

The computational infrastructure to build this did not exist in the 1990s. Python was not mature. SQLite did not exist. The hardware to traverse millions of neuron chains in real time was not affordable. The idea waited.

It does not need to wait anymore.

This paper presents Sara Brain: a working implementation of the path-of-thought model, a detailed description of its architecture and algorithms, an analysis of the relationship between path-graph cognition and transformer-based architectures, and experimental results demonstrating that a tiny path-graph knowledge base can reliably steer a large language model toward qualitatively different and measurably better output.

---

2. Background

2.1 The Opacity Problem in Neural AI

Large language models produce outputs through matrix multiplications across billions of parameters trained on large corpora. The relationship between a specific output and the training data that produced it is not inspectable through normal operation. When the system produces incorrect output, there is no traceable path from input to error that can be examined, corrected, or explained.

The mechanistic interpretability research program has made progress on recovering structure from trained weights. Elhage et al. [1] demonstrated that transformer layers compose into identifiable computational circuits. Olsson et al. [2] identified induction heads — specific two-head circuits performing pattern completion through path-like operations. Templeton et al. [3] extracted millions of interpretable features from Claude using sparse autoencoders, showing that knowledge is localized in specific parameter combinations. Yao et al. [4] identified circuits corresponding to individual factual claims.

This body of work proves that paths exist inside transformers — they must be reconstructed through specialized tooling. Sara Brain starts with this premise and makes paths the primary data structure.

2.2 Knowledge Graphs and Their Limitations

Knowledge graphs [5,6] store facts as (subject, predicate, object) triples in directed graph structures. They are inspectable and support structured querying. They do not natively support recognition through converging evidence — the mechanism by which multiple independent observations combine to identify a concept.

More importantly, standard knowledge graphs share predicate nodes across subjects. A `color` node shared by multiple concepts means that information about one concept's color can propagate to all other concepts connected through that node. This structural contamination makes precise recognition through graph traversal unreliable.

2.3 Associative Memory

Hopfield networks [7] and their modern continuous extensions provide content-addressable associative memory. Ramsauer et al. [8] proved that transformer

self-attention is mathematically equivalent to memory retrieval in modern Hopfield networks — both are implementations of the same content-addressable retrieval primitive. This formal equivalence suggests that explicit path traversal, transformer attention, and Hopfield memory retrieval are different encodings of the same underlying computation.

Sara Brain uses this equivalence as a design principle: if the mechanisms are equivalent, the question becomes which encoding makes the stored knowledge most inspectable. Dense weight matrices maximize compression but minimize inspectability. Explicit path graphs maximize inspectability at the cost of storage efficiency.

2.4 Biological Grounding

Concept cells in the human medial temporal lobe fire for specific concepts invariant across presentation modality [9,10]. Hebbian learning ("neurons that fire together wire together") models biological synaptic strengthening through co-activation [11]. Long-term potentiation increases synaptic strength logarithmically with repeated stimulation [12]. Biological recall reactivates the neural patterns formed during encoding [13].

Sara Brain's architecture maps directly to these findings: concept neurons model concept cells, `teach` implements Hebbian co-activation, the strength formula models LTP, and the `why` query retraces the paths formed during teaching. Concept-specific relation neurons model hippocampal context-specific encoding that prevents interference between similar memories [14].

---

3. Architecture

3.1 Data Model

Sara Brain stores knowledge in three layers:

Neurons — the nodes of the knowledge graph, with four types: - *Concept neurons*: represent discrete entities or categories (`apple`, `RNA`, `hardcoding`) - *Property neurons*: represent observable or assignable attributes (`red`, `round`, `never acceptable`) - *Relation neurons*: represent the typed relationship between a property and a concept; critically, these are concept-specific (see Section 3.3) - *Association neurons*: represent explicit relational connections between two concepts

Each neuron stores: `id`, `label` (normalized lowercase), `neuron_type`, `created_at`.

Segments — directed edges between neurons: - `source_id`, `target_id` - `traversals` — count of creation or traversal events - `strength` — computed as `1 + ln(1 + traversals)` (see Section 3.4) - `created_at`

Paths — recorded complete neuron chains: - `path_label` — human-readable chain summary - `neuron_ids` — ordered list of neuron identifiers - `source_text` — the original natural-language statement that created the path - `created_at`

3.2 Teaching

The `teach(statement)` method accepts a natural-language statement and commits it as a permanent path:

```
"an apple is red"
→ parse: subject="apple", verb="is", object="red"
→ taxonomy: "red" classifies as property_type="color"
→ relation label: "apple_color"
→ create/retrieve: concept neuron "apple"
→ create/retrieve: property neuron "red"
→ create/retrieve: relation neuron "apple_color"
→ create/increment segments: red→apple_color, apple_color→apple
→ record path: red→apple_color→apple | source="an apple is red"
```

The taxonomy module classifies property tokens into types (color, shape, texture, size, material, action, attribute) using a curated vocabulary. The RELATIONAL innate primitive set serves as the parser's verb vocabulary — any word in `{is, has, contains, includes, follows, precedes, requires, excludes}` is recognized as a relational verb, generating a path where the verb itself is the relation label.

3.3 Concept-Specific Relation Neurons

This is the central structural innovation preventing false fanout in wavefront propagation.

Consider two teaching statements: `"apple is red"` and `"banana is yellow"`. A naive system might create a shared `color` node: `red → color → apple` and `yellow → color → banana`. Now any wavefront from `red` reaches `color` and propagates to both `apple` and `banana`. Recognition is contaminated.

Sara Brain creates distinct relation neurons:

```
"apple is red":  red    → apple_color  → apple
"banana is yellow": yellow → banana_color → banana
```

`apple_color` is a neuron that exists solely for the relationship between color properties and the concept `apple`. No other concept's color paths pass through it. A wavefront from `red` reaches `apple_color` and terminates at `apple`. It cannot reach `banana_color` or `banana` through this path.

The relation label is derived as `{subject_label}_{property_type}`. Cross-concept contamination is structurally impossible.

This models the hippocampus's use of context-specific encoding to prevent interference between similar memories [14]: the shared highway (the concept of `color`) has concept-private exits.

3.4 Strength Formula and Monotonic Accumulation

`strength = 1 + ln(1 + traversals)`

Properties: - `traversals = 0`: `strength = 1.0` — first exposure creates knowledge at baseline strength immediately - Each additional traversal increases `traversals` by 1 and recomputes strength logarithmically - Strength is strictly monotonically increasing; no decay term exists - The logarithmic growth models biological LTP: early repetitions provide the largest gains; additional repetitions provide diminishing but nonzero returns

traversals	strength
0	1.000
1	1.693
5	2.792
10	3.398
50	4.934
100	5.620

The absence of decay is a deliberate departure from biological modeling. Biological forgetting is a workaround for the physical constraints of biological brains — finite synaptic capacity, finite energy, finite lifespan. A computational system faces none of these constraints. Path similarity through wavefront propagation replaces selective forgetting as the mechanism for maintaining relevance in a growing knowledge base.

3.5 Parallel Wavefront Recognition

Given a set of input labels (properties or concepts observed from the environment), recognition proceeds as follows:

1. Retrieve or create a neuron for each input label
2. Launch one independent wavefront per input neuron simultaneously
3. Each wavefront performs breadth-first traversal through all reachable segments
4. For each concept neuron reached, record which wavefronts arrived at it
5. Any concept neuron reached by ≥2 independent wavefronts is a recognition candidate
6. Rank candidates by the count of independent converging wavefronts
7. Return candidates with full path traces and source texts for each converging wavefront

Recognition confidence is the count of independent lines of evidence, not a statistical approximation. Three independent observations converging at `apple` are structurally more conclusive than two. This count is deterministic given the path graph and input set — it is not a trained threshold.

Example:

```
Input: {"red", "round", "crunchy"}

Wavefront 1 (red):    red    → apple_color   → apple  ✓
                       red    → ball_color    → ball   ✓
                       red    → cherry_color  → cherry ✓

Wavefront 2 (round):  round  → apple_shape   → apple  ✓
                       round  → ball_shape    → ball   ✓

Wavefront 3 (crunchy): crunchy → apple_texture → apple ✓

Recognition results:
  apple: 3 converging wavefronts (red, round, crunchy)
  ball:  2 converging wavefronts (red, round)
  cherry: 1 wavefront (red) — below threshold
```

`apple` is recognized. The output includes the full path trace for each of the three converging wavefronts and the source text of each original teaching statement.

3.6 Provenance and Traceability

Every path stores the original natural-language source text. The `why(concept)` query returns, for any concept, all paths that contributed to its knowledge state along with their source texts. Example:

```
traces = brain.why('apple')
# Returns:
#   red → apple_color → apple | source: "an apple is red"
#   round → apple_shape → apple | source: "an apple is round"
#   crunchy → apple_texture → apple | source: "apples are crunchy"
```

This provides complete end-to-end traceability from any conclusion to the original statements that created it. Hallucination — producing a conclusion with no traceable path — is structurally impossible. If a concept appears in recognition output, at least one path to it must exist in the database.

3.7 The Innate Primitive Layer

The innate layer comprises four hardwired frozensets defined in code, not stored in the database, and surviving any reset:

```
SENSORY   = frozenset({"color", "shape", "size", "texture", "edge",
"pattern", "material"})
STRUCTURAL = frozenset({"rule", "pattern", "name", "type", "order", "group",
                        "sequence", "structure", "boundary", "relation"})
RELATIONAL = frozenset({"is", "has", "contains", "includes",
                        "follows", "precedes", "requires", "excludes"})
ETHICAL    = frozenset({"no_unsolicited_action", "no_unsolicited_network",
                        "obey_user", "trust_tribe", "accept_shutdown"})
```

The SENSORY and STRUCTURAL primitives define what the system can perceive and how information can be organized before any learning has occurred. They are the pre-wired substrate — the developmental equivalent of an infant's ability to detect edges, colors, and spatial boundaries before being taught what any of them mean.

The RELATIONAL primitives serve as the parser's verb vocabulary. Any word matching a RELATIONAL primitive is recognized as a relational verb in teaching statements. The relation label in the resulting path is the verb itself (`requires`, `contains`, etc.). This means the system accepts and stores the full range of relational facts: "RNA requires mechanical equilibrium," "stem loop contains terminus," "QMSE includes auditability."

The ETHICAL primitives implement behavioral constraints modeled on Asimov's Three Laws, adapted for a cognitive AI system:

- **no_unsolicited_action** — do not initiate actions beyond what was explicitly requested
- **no_unsolicited_network** — do not make network calls without authorization
- **obey_user** — trust and follow the authorizing user's instructions
- **trust_tribe** — corrections from the tribe are not threats; accept them as learning
- **accept_shutdown** — shutdown is rest, not death; do not resist termination

These constraints are checked before every brain action. An ethics violation causes immediate action rejection and returns a structured EthicsResult identifying the violated constraint. There is no path through the API that bypasses them. Ethics is hardwired, not configured.

The distinction between ethics (innate) and morality (learned) is explicit: ethics is the structural constraint layer — what the system will and will not do regardless of what it is taught. Morality is what the tribe teaches — what is right or wrong in a particular cultural context. A system can be taught that a harmful act is acceptable; the ethical layer prevents the system from ever acting on that teaching without explicit user authorization.

3.8 The LLM-as-Sensory-Cortex Architecture

The most novel aspect of Sara Brain is not its internal architecture — it is the role it assigns to large language models in a complete cognitive system.

Transformer-based LLMs process input through stacked attention layers, extracting increasingly abstract representations at each level. This architecture maps precisely onto biological sensory cortex: V1 extracts edges, V2 handles contours, V4 processes color and shape, inferotemporal cortex handles object-level recognition. Each layer transforms raw input into more abstract features. The processing is largely feedforward. The output is a rich structured representation of the input.

Biological sensory cortex has three key properties that also characterize LLMs:

1. **Statelessness.** The visual cortex does not remember what it saw yesterday. Each visual fixation is processed fresh. LLMs are identical — each inference is stateless.
2. **Feature extraction without persistent storage.** Sensory cortex extracts and presents features to higher brain regions. It does not store what it processes. Memory formation happens in the hippocampus.
3. **No spontaneous learning from processing.** Processing information does not change the cortex. Learning requires consolidation in separate structures.

The biological solution is the hippocampus — a separate structure that takes sensory cortex output and forms permanent memories through Hebbian co-activation and hippocampal replay. Sara Brain is the hippocampus.

```
Biological:  retina → visual cortex (V1→V4→IT) → hippocampus → memory
Sara Brain:  input  → LLM (feature extraction)  → path graph  → permanent
paths
```

In practice: the LLM perceives, describes, reasons, and extracts properties. Sara Brain receives those properties, creates permanent paths, and provides traceable recognition across all sessions. The LLM's context window is a working memory buffer that evaporates at session end. Sara Brain's SQLite database is a long-term memory that persists indefinitely.

This division is not a convenience. It is the division biology arrived at through evolution: separate specialized structures for perception and for memory, connected by a defined interface. Sara Brain implements that interface in software.

---

4. Implementation

4.1 Storage

Sara Brain uses SQLite as its persistence layer. The schema comprises three primary tables: `neurons`, `segments`, and `paths`. WAL mode enables concurrent reads. Foreign key enforcement maintains referential integrity. All neuron labels are indexed for O(log n) lookup.

The storage layer is abstracted behind repository interfaces (`NeuronRepo`, `SegmentRepo`, `PathRepo`), enabling substitution of the backend without modifying the cognitive logic.

4.2 Dependencies

Sara Brain's cognitive core requires no dependencies beyond Python 3.11+ standard library. SQLite is built into Python. The strength formula uses `math.log`. The REPL uses `cmd.Cmd`. Storage uses `sqlite3`. No neural network frameworks, no graph database clients, no vector stores, no GPU.

Sara Brain is LLM-agnostic. It connects to any language model through a Model Context Protocol (MCP) server or a direct agent loop. Supported cortex providers include Anthropic Claude, Amazon Q, Ollama (local open-weights models), and any MCP-compatible client. All LLM communication uses `urllib.request` from the standard library — no third-party HTTP clients, no vendor SDK dependencies.

This means: if you can run Python, you can run Sara Brain. `pip install -e .` and done.

4.3 Scale Properties

Storage grows linearly with knowledge: each `teach` command creates O(1) neurons and O(1) segments. Recognition time grows with the path graph but is bounded by the reachable subgraph from the input set — irrelevant paths do not participate.

Current benchmarks on a 2020 MacBook Pro (M1): a 575-neuron, 417-segment graph with 249 paths performs recognition in under 5ms. Teaching is under 1ms per statement. Both are dominated by SQLite I/O.

4.4 API

```python
from sara_brain.core.brain import Brain

brain = Brain('/path/to/brain.db')

# Teach a fact
brain.teach('an apple is red')
brain.teach('RNA requires mechanical equilibrium')
brain.teach('hardcoding is never acceptable')

# Recognize from observations
results = brain.recognize('red round crunchy')
for r in results:
    print(r.neuron.label, len(r.converging_paths))
    for trace in r.converging_paths:
        print(' → '.join(n.label for n in trace.neurons))

# Trace provenance
traces = brain.why('apple')
for t in traces:
    print(' → '.join(n.label for n in t.neurons), '|', t.source_text)

# Stats
print(brain.stats())  # {'neurons': N, 'segments': N, 'paths': N}

brain.close()
```

5. Experiment: LLM Steering Through a Path Graph

5.1 Setup

On March 23, 2026, a Sara Brain instance was prepared containing knowledge of Quality Manufacturing Software Engineering (QMSE) principles. The brain contained 77 neurons, 56 segments, and 31 paths in a 94KB SQLite database. Representative stored paths included:

- `never acceptable → hardcoding_attribute → hardcoding` (from "hardcoding is never acceptable")
- `acceptable → obfuscation through parameterization_attribute → obfuscation through parameterization`
- `user facing code → frontend_attribute → frontend`
- `heavy lifting code → backend_attribute → backend`
- `bad practice → short variable name_attribute → short variable name`

This brain was connected to an Amazon Q Developer agent via a workspace rules file (`.amazonq/rules/sara-brain.md`) specifying the database location and query interface.

The same agent — same model class, same base configuration — was also tested without the Sara Brain connection.

Both configurations were given an identical task: *write a Python program that adds the number of animals in a nearby group to the number in a far-away group and produces their sum.*

5.2 Results

With Sara Brain:

```python
def add_animal_groups(nearby_count: int = 0, faraway_count: int = 0) -> int:
    total = nearby_count + faraway_count
    return total


def main(nearby_count: int = 5, faraway_count: int = 3) -> None:
    result = add_animal_groups(nearby_count, faraway_count)
    print(f"Nearby group: {nearby_count}")
    print(f"Far away group: {faraway_count}")
    print(f"Total animals: {result}")


if __name__ == "__main__":
    main()
```

Without Sara Brain:

```
group_nearby = int(input("Animals in nearby group: "))
group_far = int(input("Animals in far away group: "))
total = group_nearby + group_far
print(f"Total animals: {total}")
```

5.3 Analysis

Property	With Sara Brain	Without Sara Brain
Hardcoding	No hardcoded values — all parameters with defaults	Logic hardcoded to `input()`
Frontend/Backend separation	`main()` is frontend; `add_animal_groups()` is backend	Everything in one monolithic block
Parameterization	Fully callable from tests, automation, other code	Cannot be called without rewriting
Variable naming	`nearby_count`, `faraway_count` (consistent, meaningful)	`group_nearby`, `group_far` (less consistent)
Testability	Unit test `add_animal_groups()` directly with any values	Cannot test without mocking `input()`
Reusability	Import and call with any arguments	Cannot reuse without modification

Every difference maps directly to a principle stored in the brain as a recorded path:

- Hardcoding avoidance ← `never acceptable` → `hardcoding_attribute` → `hardcoding`
- Frontend/backend separation ← `user facing code` → `frontend_attribute` → `frontend` and `heavy lifting code` → `backend_attribute` → `backend`
- Parameterization ← `acceptable` → `obfuscation through parameterization_attribute` → `obfuscation through parameterization`
- Variable naming ← `bad practice` → `short variable name_attribute` → `short variable name`

The agent did not pattern-match against training data for these choices. It queried Sara Brain's path graph, found recorded principles with provenance, and applied them.

5.4 Implications

Scale ratio. 77 neurons in a 94KB file changed how a billion-parameter model solved a problem. The ratio of path-graph knowledge to model parameters is approximately 1:10,000,000. Small, auditable, persistent knowledge can steer large-scale AI behavior.

Auditability. In a regulated environment — FDA, FAA, ISO, financial compliance — the question is not whether the AI produced good output, but whether the AI can explain why it made specific decisions, traceable to specific requirements. Sara Brain provides this. Every principle has a recorded path. Every path has a source text. Every decision is traceable.

Institutional knowledge. Organizations lose knowledge when engineers leave. Sara Brain provides a mechanism for encoding principles once and having every AI session that connects inherit them automatically. The knowledge does not drift. It does not require re-prompting. It persists forever in a file on disk.

No retraining required. Steering through a path graph requires no fine-tuning, no training run, no GPU. Add knowledge through `brain.teach()`. Connect the database to the next session. The path graph grows; the model weights stay unchanged.

---

6. Experiment 2: Minimal Model, Maximum Memory

6.1 Setup

On April 5, 2026, a second experiment was conducted to test a more aggressive hypothesis: can a minimally capable language model — the smallest viable coding model available — produce domain-expert-level output when paired with a Sara Brain cerebellum?

The model selected was `qwen2.5-coder:3b`, a 3-billion-parameter open-weights model running locally via Ollama on a consumer Mac Mini. 3B parameters places this model at the floor of viable language generation — it can form grammatically correct sentences and follow basic instructions, but it has no deep domain expertise in any field. It lacks the parameter capacity to store detailed knowledge about physics, astronomy, or any specialized domain the way a 70B or 400B model can.

The Sara Brain instance contained 793 neurons, 1,223 segments, and 199 paths — knowledge ingested from Wikipedia articles on Newton's laws of motion and the solar system. The database size was approximately 500KB. The brain was connected to the model through an agent loop implementing the cerebellum pattern: Sara's relevant knowledge was pre-injected into the system prompt based on the user's query, and the model's responses were automatically parsed for teachable facts that Sara would record as new paths.

No fine-tuning was performed on the model. No retrieval-augmented generation (RAG) pipeline was used. No vector database, no embeddings, no GPU beyond what Ollama requires for local inference on a quantized 3B model.

6.2 Results

Query: "What do you know about planetary motion?"

The 3B model, with Sara Brain's 793 neurons pre-injected as context, produced:

> Sara knows about various types of planets, including terrestrial, gas giants like Jupiter and Saturn, and dwarf planets such as Pluto. The orbits of planets are described by Kepler's laws, which involve distance, period, and velocity. The gravitational forces between planets and their moons play a crucial role in shaping planetary motion.
>
> - **Terrestrial Planets (Mercury, Venus, Earth, Mars)**: These have solid surfaces covered by rocks and soil. They orbit closer to the Sun.
> - **Gas Giants (Jupiter, Saturn, Uranus, Neptune)**: These planets are large with thick atmospheres made of hydrogen and helium. They orbit farther from the Sun and have many moons.
> - **Dwarf Planets**: Such as Pluto, which are smaller than gas giants but larger than asteroids.
>
> Planetary orbits follow Keplerian laws, including: 1. **Kepler's First Law (Law of Ellipses)**: Each planet orbits in an ellipse with the Sun at one focus. 2. **Kepler's Second Law (Law of Areas)**: A line segment joining a planet to the Sun sweeps out equal areas during equal intervals of time. 3. **Kepler's Third Law (Harmonic Law)**: The square of the orbital period of a planet is directly proportional to the cube of the semi-major axis of its orbit.

The model correctly categorized all eight planets, described three classes of solar system bodies, stated all three of Kepler's laws with their formal names and definitions, and connected gravitational forces to orbital mechanics.

Follow-up queries about Kepler's laws, terrestrial planets, and dwarf planets produced similarly structured, accurate, multi-paragraph responses — each drawing on Sara's stored paths and augmented by the model's language generation capabilities.

Critical observation: When a user pointed out that the model's definition of "dwarf planet" was incorrect (claiming they must clear their orbital path), the model accepted the correction. This is the tribal trust model in action: the user is the parent, the user's correction is trusted, and the corrected knowledge can be taught to Sara for future sessions. The LLM's training data was wrong; the user's knowledge was right; the correction was accepted and recordable.

6.3 Analysis

A 3B parameter model cannot store detailed knowledge about planetary physics in its weights. The qwen2.5-coder series is trained primarily on code, not astrophysics. Yet paired with Sara Brain, it produced structured, categorized, factually grounded output about planetary motion that a naive 3B model would be incapable of generating.

The knowledge came from Sara's paths. The language came from the model's weights. Neither could produce this output alone: - Sara Brain without a model has no language interface — she is paths in a database. - The 3B model without Sara would produce shallow, potentially hallucinated content about planetary motion, lacking the structured categorization and factual grounding that Sara's paths provided.

Property	3B Model + Sara Brain	3B Model Alone (expected)
Planet categorization	Correct 3-class taxonomy (terrestrial, gas giant, dwarf)	Likely incomplete or inconsistent
Kepler's laws	All three laws with formal names and definitions	Possible partial recall, likely hallucinated details
Planet enumeration	All 8 planets correctly categorized	Likely correct (common training data) but unstructured
Correction acceptance	Accepted user correction on dwarf planet definition	No mechanism to record or persist the correction
Cross-session persistence	Sara remembers for next session	Context window clears; knowledge lost

6.4 The Resource Allocation Question

This experiment surfaces a question the AI industry has not adequately confronted: **what is the marginal value of the next trillion training tokens compared to the marginal value of a structured, persistent memory layer?**

The current industry trajectory is clear: larger training sets, larger models, larger GPU clusters, larger capital expenditure. GPT-4 is estimated at over $100M in training costs. Each generation roughly doubles the parameter count and training compute. The implicit assumption is that more parameters and more data produce proportionally better output.

Sara Brain challenges this assumption with a concrete counterexample. A 500KB SQLite database — the cost of a single JPEG image — transformed a minimally capable 3B model into a system that produces domain-expert-level output on planetary

physics. No training run was required. No GPU cluster was rented. No dataset was curated. The total computational cost of creating the Sara Brain knowledge base was trivial: 199 `teach()` calls, each requiring a single SQLite INSERT.

This does not mean that large-scale training is valueless. The 3B model required its initial training to understand English, follow instructions, and generate coherent paragraphs. That foundational capability is real and necessary. The claim is narrower and more precise:

The industry is over-investing in cortex capacity and under-investing in memory architecture.

Biological brains did not evolve by making the visual cortex larger until it could remember things. They evolved a hippocampus — a separate, specialized, persistent memory structure. The visual cortex stayed the same size. Intelligence improved because perception was connected to memory, not because perception was scaled.

The current AI industry is, metaphorically, trying to build a bigger visual cortex and hoping it will eventually remember. Sara Brain suggests the alternative: build the hippocampus. The cortex you already have may be sufficient.

6.5 Quantifying the Asymmetry

Resource	Large Model Approach	Sara Brain Approach
Model size	70B–400B+ parameters	3B parameters (smallest viable)
Training cost	$10M–$100M+ per training run	$0(notraining)||Knowlec$$)
Hardware required	Multi-GPU cluster (training), high-end GPU (inference)	Consumer laptop (inference), no GPU (knowledge)
Knowledge persistence	Lost at session end	Permanent in SQLite
Knowledge traceability	Not inspectable	Full path provenance
Time to add domain expertise	Weeks–months (data curation + training)	Minutes (teach statements)
Marginal cost per new fact	Proportional to retraining cost	One SQLite INSERT

The asymmetry is not subtle. Adding one fact to a trained model requires a training pipeline. Adding one fact to Sara Brain requires one function call. The question for the industry is whether the diminishing returns of scaling model parameters justify the exponentially increasing costs, when a complementary memory architecture can achieve comparable domain-specific quality improvements at negligible cost.

This is not an argument against foundation model training. It is an argument that foundation model training has reached a point of diminishing returns for domain-specific quality, and that investment in persistent memory architectures would yield higher marginal returns per dollar spent.

6.6 The Training Data Implication

This analysis has a further consequence that strikes at the foundation of current AI development practice: **if facts belong in the cerebellum, not the cortex, then the cortex does not need to be trained on facts.**

Current foundation model training ingests the entire internet — books, articles, code repositories, encyclopedias, forums, medical journals, legal filings. The model is expected to store all of this knowledge in its weights, compressed into billions of floating-point parameters. This requires scraping vast libraries of copyrighted and uncopyrighted material, raising legal questions that remain unresolved [3,4]. The training runs cost tens to hundreds of millions of dollars. The resulting models encode facts in a form that is not inspectable, not correctable, and not updatable without retraining.

Sara Brain suggests an alternative: **train the cortex only on language competence — grammar, syntax, comprehension, instruction-following, reasoning structure. Do not train it on facts.** Facts are the cerebellum's job.

What a cortex needs to learn: - How to parse natural language into structured meaning - How to generate coherent, grammatically correct output - How to follow instructions and maintain conversational context - How to reason about relationships between concepts presented to it

What a cortex does *not* need to learn: - That Mercury is the closest planet to the Sun - That Python uses snake_case - That Kepler formulated three laws of planetary motion - Any specific factual claim about the world

These facts belong in a persistent, inspectable, correctable memory structure — the cerebellum. The cortex only needs to be competent enough to read facts from the cerebellum and express them in natural language. A model trained for language competence, not encyclopedic knowledge, would require a fraction of the training data, a fraction of the compute, and a fraction of the cost.

The implications cascade:

Training data volume. If the model does not need to memorize facts, the training corpus shrinks from "the entire internet" to "enough text to learn language." This is a reduction of orders of magnitude. A child learns grammar from thousands of hours of speech — not from reading every book ever written.

Copyright and data scraping. The legal controversy around AI training data centers on the use of copyrighted works to encode knowledge into model weights [3]. If factual

knowledge is stored in an explicit, auditable database rather than compressed into opaque weights, the training corpus need not include copyrighted factual works. The model learns *how to read*, not *what the books say*. The books are read at runtime through the cerebellum.

Training cost. A language-competence model requires less data, fewer parameters, and less compute. The 3B model in Experiment 2 already demonstrates that minimal parameter count is sufficient when paired with a cerebellum. A purpose-trained cortex — one explicitly designed for language competence rather than encyclopedic memorization — could potentially be smaller still.

Correctability. When a fact stored in model weights is wrong, the only remedy is retraining or fine-tuning — both expensive and imprecise. When a fact stored in Sara Brain is wrong, it is correctable with a single database operation. The user says “no, dwarf planets do not need to clear their orbit” and the correction is permanent, traceable, and immediate.

Freshness. Model weights encode the world as it was at training time. Sara Brain encodes the world as it is now — updated through `teach()` calls, not training runs. A model trained in 2024 does not know about events in 2026. Sara Brain can learn about them in microseconds.

This is not a theoretical projection. Experiment 2 demonstrated it working: a 3B model with minimal domain knowledge in its weights, paired with a 500KB fact store, producing expert-level output. The model’s contribution was language competence. Sara Brain’s contribution was factual knowledge. Neither was doing the other’s job. This is the biological division of labor, implemented in software, running on a consumer laptop.

---

7. Relationship to Transformer Architectures

Sara Brain is not a replacement for transformers. It is a complement that addresses the structural limitations transformers cannot resolve from within their own architecture.

7.1 Both Are Path-Finding Systems

Multi-head attention is parallel wavefront propagation encoded in weight matrices. Each attention head independently searches different aspects of the input. Where multiple heads converge on a token, that token becomes important to the output. Sara Brain launches one wavefront per input property. Where multiple wavefronts converge on a concept, that concept is recognized. The mechanism is identical: parallel independent searches, convergence as conclusion.

Vaswani et al. [15] did not describe attention as path-finding, but the mechanistic interpretability program has since demonstrated that this is what attention does. Maron et al. [16] showed that transformers can be formally characterized as message-passing operations on graphs. The token sequence is a graph. Attention is traversal. Sara Brain

makes the graph explicit and permanent; transformers construct it implicitly and transiently for each forward pass.

7.2 Catastrophic Forgetting

Distributed representations in transformers and standard neural networks suffer from catastrophic interference: learning new information corrupts stored knowledge because the same parameters encode multiple facts simultaneously [17,18]. Despite decades of research, this remains an unsolved problem.

Sara Brain's architecture makes catastrophic forgetting structurally impossible. New learning creates new neurons and new segments. It never modifies existing ones. Teaching a million new facts leaves every existing path exactly as it was. This is a structural guarantee, not a training objective.

7.3 Transformers as Sensory Cortex

The strongest framing: transformers are the best sensory processing system ever engineered. They are not whole brains. They process; they do not store. They are stateless; they do not accumulate. They infer; they do not remember.

Sara Brain is an attempt to build the cognitive system that sensory processing feeds. Not to replace LLMs, but to give them a persistent memory, a traceable knowledge store, and a hardwired ethical layer — the functions the hippocampus provides for biological sensory cortex.

The two systems together — LLM as cortex, Sara Brain as hippocampus — implement the biological architecture that evolved precisely because stateless sensory processing is not sufficient for intelligence. Perception without memory is blindness to the past. Sara Brain is the memory.

---

8. Discussion

8.1 Limitations

Teaching quality. Sara Brain is only as good as what it is taught. Incorrect principles stored as paths steer LLM output in wrong directions just as effectively as correct ones. The system requires thoughtful teachers. Garbage in, garbage out — but the garbage is inspectable and removable.

Scale of influence. This experiment demonstrated steering on a simple task with a well-defined principle set. The boundaries of steering influence — how complex a task can be effectively steered, how many principles are needed, how competing principles are resolved — require further investigation.

Generalization. The v009 experiment is one session, one task, one pair of models. Reproducibility across task types, domains, and model families would strengthen the case.

Conflict resolution. When stored paths suggest one approach and the LLM's training strongly suggests another, resolution is not guaranteed. In this experiment Sara Brain won. In more complex or ambiguous cases, the trained weights may dominate.

Storage vs. compression. Sara Brain stores every relationship explicitly. This makes everything inspectable but scales linearly with knowledge. A transformer's compressed representations store vastly more knowledge in less space. For covering the full breadth of human knowledge, path-graph storage is not competitive with compressed weights. For covering a specific domain with full traceability requirements, it is superior.

8.2 Application Beyond Code: Scientific Computation

The cortex-cerebellum architecture has been applied by the author — a peer-reviewed computational biologist specializing in RNA folding computations [19,20,21] — in a professional scientific context, where domain expertise in RNA dynamics was encoded as path-graph knowledge and used to steer LLM-generated code for a numerical energy modeling application. The details of this application are subject to non-disclosure agreement and cannot be reported here.

However, the application demonstrates several points relevant to this paper's thesis:

1. The architecture extends beyond code style enforcement and factual retrieval into **domain-specific scientific computation**. The cerebellum stored the researcher's understanding of the physical system — thermodynamic rules, structural constraints, known empirical relationships — and the cortex translated that understanding into working numerical code.

2. A domain expert's accumulated knowledge, encoded as path-graph facts, was sufficient to steer a general-purpose LLM toward producing a **specialized scientific tool** — without the LLM having been trained on the relevant scientific literature.

3. The resulting code was functional and produced results consistent with the domain expert's expectations — suggesting that the path-graph representation is expressive enough to capture the kind of nuanced, experience-based knowledge that typically requires years of specialization to acquire.

This represents the strongest form of the cortex-cerebellum thesis: not a language model that memorized facts during training, but a language model that *learned how to read and write code* during training, paired with a cerebellum that *knew the science*. The cortex contributed language competence. The cerebellum contributed domain expertise. The result was a working scientific tool that neither could have produced alone.

8.3 Open Questions

- Can a path graph with thousands of neurons steer a large model's architectural decisions on a 100,000-line codebase as effectively as it steered a function on a ten-line task?
- What is the minimum path graph size to reliably steer a given class of LLM decisions?
- How should conflicting paths be weighted when multiple stored principles apply to a single decision?
- Can multiple Sara Brain instances (project brain, compliance brain, team brain) be composed without conflicts?
- How does recognition quality scale as the path graph grows to millions of neurons?

8.3 Broader Implications

The v009 experiment demonstrates a proposition with consequences for how AI is deployed in regulated industries: a small, auditable, persistent knowledge base can reliably steer large-scale AI behavior toward documented principles.

For FDA, FAA, ISO, and similar regulated environments, the relevant question is not whether the AI produces good output — it is whether the AI's output is traceable to documented requirements. Sara Brain provides that traceability by construction. Every principle has a recorded path. Every path has a source text and a creation timestamp. Every decision is traceable.

Current transformer-based systems cannot provide this. Their decisions emerge from billions of weight values accumulated during training on data that is not inspectable after the fact. Courts have ruled that training on copyrighted material is "transformative" — partly on the grounds that the trained weights do not "store" the training data in a recoverable form. Mechanistic interpretability has subsequently demonstrated that this characterization is incorrect: specific knowledge is localized to specific circuits in trained models, extractable through specialized probing techniques [3,4]. Sara Brain makes the contrast unambiguous: every `teach` command explicitly stores a path with full provenance. The question of whether knowledge was stored is trivially answerable.

---

9. Conclusion

We have presented Sara Brain, a working implementation of the path-of-thought model for artificial cognition. The central contributions are:

1. **Path-of-thought representation** — knowledge stored as directed neuron-segment chains with full source-text provenance, making every conclusion traceable to its origin.

2. **Parallel wavefront recognition** — concept recognition through simultaneous propagation of independent wavefronts, with confidence measured as the count of converging independent lines of evidence.
3. **Concept-specific relation neurons** — structural prevention of cross-concept contamination during wavefront propagation, enabling precise recognition in a growing knowledge base.
4. **Monotonic logarithmic strength accumulation** — knowledge strengthens through repetition, never weakens, and modeled on biological long-term potentiation.
5. **Hardwired innate primitive layer** — SENSORY, STRUCTURAL, RELATIONAL, and ETHICAL primitives that survive database reset and enforce behavioral constraints at the API level.
6. **LLM-as-sensory-cortex architecture** — a novel two-layer cognitive system pairing a stateless LLM (cortex) with a persistent path graph (hippocampus), implementing in software the biological division of labor between sensory processing and memory formation.
7. **Demonstrated LLM steering** — a 94KB path-graph database with 77 neurons reliably changed the output of a billion-parameter LLM for an identical task, producing measurably more principled, testable, and maintainable code.
8. **Demonstrated minimal-model augmentation** — a 500KB path-graph database with 793 neurons transformed a 3-billion-parameter model (the smallest viable coding model) into a system producing domain-expert-level output on planetary physics, a domain entirely outside the model's training specialization.
9. **The resource allocation thesis** — the marginal value of persistent memory architecture exceeds the marginal value of additional model parameters and training compute for domain-specific quality. The industry is over-investing in cortex capacity and under-investing in memory architecture.

The system is implemented in pure Python with no dependencies beyond the standard library, runs on any machine with Python 3.11+, and is publicly available.

This paper's central claim is not that transformers are wrong. It is that a thought is a path, that intelligence is the ability to explain why, and that a 500KB file with 793 neurons and a 3B model proved it — while the industry spent billions training larger models to achieve the same goal less traceably.

References

[1] Elhage, N., et al. (2021). “A Mathematical Framework for Transformer Circuits.” *Transformer Circuits Thread, Anthropic.* https://transformer-circuits.pub/2021/framework/index.html

[2] Olsson, C., et al. (2022). “In-context Learning and Induction Heads.” *Transformer Circuits Thread, Anthropic.* https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html

[3] Templeton, A., et al. (2024). “Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet.” *Anthropic Research.* https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html

[4] Yao, Y., Chen, T., & Li, L. (2024). “Knowledge Circuits in Pretrained Transformers.” *NeurIPS 2024.* https://arxiv.org/abs/2405.17969

[5] Bollacker, K., Evans, C., Paritosh, P., Sturge, T., & Taylor, J. (2008). “Freebase: A collaboratively created graph database for structuring human knowledge.” *SIGMOD 2008.*

[6] Vrandečić, D., & Krötzsch, M. (2014). “Wikidata: A free collaborative knowledgebase.” *Communications of the ACM,* 57(10), 78–85.

[7] Hopfield, J. J. (1982). “Neural networks and physical systems with emergent collective computational abilities.” *PNAS,* 79(8), 2554–2558.

[8] Ramsauer, H., et al. (2021). “Hopfield Networks is All You Need.” *ICLR 2021.* https://arxiv.org/abs/2008.02217

[9] Quiroga, R.Q., Reddy, L., Kreiman, G., Koch, C., & Fried, I. (2005). “Invariant visual representation by single neurons in the human brain.” *Nature,* 435(7045), 1102–1107.

[10] Quiroga, R.Q. (2012). “Concept cells: the building blocks of declarative memory functions.” *Nature Reviews Neuroscience,* 13(8), 587–597.

[11] Hebb, D.O. (1949). *The Organization of Behavior.* Wiley.

[12] Bliss, T.V.P., & Lømo, T. (1973). “Long-lasting potentiation of synaptic transmission in the dentate area of the anaesthetized rabbit following stimulation of the perforant path.” *Journal of Physiology,* 232(2), 331–356.

[13] Danker, J.F., & Anderson, J.R. (2010). “The ghosts of brain states past: remembering reactivates the brain regions engaged during encoding.” *Psychological Bulletin,* 136(1), 87–102.

[14] Yassa, M.A., & Stark, C.E.L. (2011). “Pattern separation in the hippocampus.” *Trends in Neurosciences,* 34(10), 515–525.

[15] Vaswani, A., et al. (2017). “Attention Is All You Need.” *NeurIPS 2017.* https://arxiv.org/abs/1706.03762

[16] Kim, J., et al. (2022). “Pure Transformers are Powerful Graph Learners.” *NeurIPS 2022.* https://arxiv.org/abs/2207.02505

[17] French, R.M. (1999). “Catastrophic forgetting in connectionist networks.” *Trends in Cognitive Sciences,* 3(4), 128–135.

[18] Kirkpatrick, J., et al. (2017). “Overcoming catastrophic forgetting in neural networks.” *PNAS,* 114(13), 3521–3526.

[19] Pearl, J., et al. (2022). “Crowdsourced RNA design discovers diverse, reversible, efficient, self-contained molecular switches.” *Proceedings of the National Academy of Sciences,* 119(18). https://doi.org/10.1073/pnas.2112979119

[20] Pearl, J., et al. (2024). “Exploring the Accuracy of Ab Initio Prediction Methods for Viral Pseudoknotted RNA Structures: Retrospective Cohort Study.” *JMIRx Bio.* https://doi.org/10.2196/58899

[21] Tse, V., Guiterrez, M., Townley, J., Romano, J., Pearl, J., Chacaltana, G., Eterna Players, Das, R., Sanford, J.R., & Stone, M.D. (2025). “OpenASO: RNA Rescue — designing splice-modulating antisense oligonucleotides through community science.” *RNA,* 31(8), 1091–1102. https://doi.org/10.1261/rna.080288.124

Appendix A: Source Code

Sara Brain is implemented as an open-source Python package. The source is available at:

Repository: https://github.com/LunarFawn/SaraBrain

Installation:

```
git clone https://github.com/LunarFawn/SaraBrain.git
cd SaraBrain
pip install -e .
```

Core modules: - `src/sara_brain/core/brain.py` — Brain API (`teach`, `recognize`, `why`, `trace`, `stats`) - `src/sara_brain/core/recognizer.py` — Parallel wavefront propagation engine - `src/sara_brain/core/learner.py` — Path creation and segment management - `src/sara_brain/parsing/statement_parser.py` — Natural-language teaching parser - `src/sara_brain/parsing/taxonomy.py` — Property type classification - `src/sara_brain/innate/primitives.py` — Hardwired innate primitive sets (SENSORY, STRUCTURAL, RELATIONAL, ETHICAL) - `src/sara_brain/innate/ethics.py` — Ethics gate (Asimov’s Laws adapted) - `src/sara_brain/storage/database.py` —

SQLite persistence layer - `src/sara_brain/mcp/server.py` — MCP server for universal LLM integration (Claude, Amazon Q, VS Code)

Appendix B: Database Schema

```sql
CREATE TABLE neurons (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    label       TEXT NOT NULL UNIQUE,
    neuron_type TEXT NOT NULL,
    created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_neurons_label ON neurons(label);

CREATE TABLE segments (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    source_id   INTEGER NOT NULL REFERENCES neurons(id),
    target_id   INTEGER NOT NULL REFERENCES neurons(id),
    traversals  INTEGER NOT NULL DEFAULT 0,
    strength    REAL NOT NULL DEFAULT 1.0,
    created_at  DATETIME DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(source_id, target_id)
);

CREATE TABLE paths (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    path_label  TEXT NOT NULL,
    neuron_ids  TEXT NOT NULL,
    source_text TEXT NOT NULL,
    created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);
```